Data Structures Home Work Set 6

Doubly Linked Lists and Trees

Register No: 2025115058

Name: Sri Senthil Adavan K

Q1. Simulate the undo and redo operations in a word-processing application using a doubly linked list.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
struct State { char text[100]; struct State *prev, *next; };
struct State *make(char *s) { struct State *p=malloc(sizeof(*p)); strcpy(p->text,s); p->prev=p->next=NULL; return p; }
int main(void) {
   struct State *current=make(""); char command[20], word[50];
   int choice;
   do {
      printf("\n1.Type word 2.Undo 3.Redo 4.Show 0.Exit: "); scanf("%d",&choice);
      if(choice==1) {
         printf("Enter word: "); scanf("%49s",word);
         struct State *p=make(current->text);
         if(strlen(p->text)>0) strcat(p->text," "); strcat(p->text,word);
         current->next=NULL; p->prev=current; current->next=p; current=p;
      } else if(choice==2 && current->prev) current=current->prev;
      else if(choice==3 && current->next) current=current->next;
      else if(choice==4) printf("Text: %s\n",current->text);
   } while(choice!=0);
   return 0;
}
```

Output:

```
1.Type word 2.Undo 3.Redo 4.Show 0.Exit: 1
Enter word: Hello
1.Type word 2.Undo 3.Redo 4.Show 0.Exit: 1
Enter word: World
1.Type word 2.Undo 3.Redo 4.Show 0.Exit: 2
1.Type word 2.Undo 3.Redo 4.Show 0.Exit: 4
Text: Hello
1.Type word 2.Undo 3.Redo 4.Show 0.Exit: 3
1.Type word 2.Undo 3.Redo 4.Show 0.Exit: 4
Text: Hello World
```

Q2. Given two sorted lists, L1 and L2, compute L1 intersection L2 and L1 union L2 using basic list operations.

Code:

```
#include <stdio.h>
int main(void) {
   int a[50],b[50],u[100],in[50],n,m,i=0,j=0,k=0,t=0;
   printf("Enter size of L1 and L2: "); scanf("%d%d",&n,&m);
   printf("Enter sorted L1: "); for(i=0;i<n;i++) scanf("%d",&a[i]);
   printf("Enter sorted L2: "); for(i=0;i<m;i++) scanf("%d",&b[i]);
   i=j=0;
   while(i<n && j<m) {
      if(a[i]<b[j]) u[k++]=a[i++];
```

```
        else if(a[i]>b[j]) u[k++]=b[j++];
        else { in[t++]=a[i]; u[k++]=a[i]; i++; j++; }
    }
    while(i<n) u[k++]=a[i++]; while(j<m) u[k++]=b[j++];
    printf("Intersection: "); for(i=0;i<t;i++) printf("%d ",in[i]);
    printf("\nUnion: "); for(i=0;i<k;i++) printf("%d ",u[i]); printf("\n");
    return 0;
}
```

Output:

```
Enter size of L1 and L2: 5 4
Enter sorted L1: 1 2 4 6 8
Enter sorted L2: 2 3 6 9
Intersection: 2 6
Union: 1 2 3 4 6 8 9
```

Q3. Swap two elements by adjusting only the pointers, not the data, using a doubly linked list.

Code:

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *prev,*next; };
void swap(struct Node **head, struct Node *x, struct Node *y) {
    struct Node *p;
    if(x==y) return;
    if(x->next==y) { /* x before y */
        if(x->prev) x->prev->next=y; else *head=y;
        if(y->next) y->next->prev=x;
        x->next=y->next; y->prev=x->prev; y->next=x; x->prev=y; return;
    }
    if(y->next==x) { /* y before x */
        if(y->prev) y->prev->next=x; else *head=x;
        if(x->next) x->next->prev=y;
        y->next=x->next; x->prev=y->prev; x->next=y; y->prev=x; return;
    }
    if(x->prev) x->prev->next=y; else *head=y;
    if(x->next) x->next->prev=y;
    if(y->prev) y->prev->next=x; else *head=x;
    if(y->next) y->next->prev=x;
    p=x->prev; x->prev=y->prev; y->prev=p;
    p=x->next; x->next=y->next; y->next=p;
}
int main(void) {
    int a[]={10,20,30,40},i; struct Node *h=NULL,*tail=NULL,*p,*x=NULL,*y=NULL;
    for(i=0;i<4;i++){ p=malloc(sizeof(*p)); p->data=a[i]; p->next=NULL; p->prev=tail; if(tail)tail->next=p;else h=p; tail=p; if(i==1)x=p;if(i==3)y=p; }
    swap(&h,x,y); printf("After swapping 20 and 40: ");
    for(p=h;p;p=p->next) printf("%d ",p->data); printf("\n"); return 0;
}
```

Output:

```
After swapping 20 and 40: 10 40 30 20
```

Q4. Implement a self-adjusting list. When an element is found, move it to the front without changing the relative order of the other elements.

Code:

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *prev,*next; };
```

```
int main(void) {
    int a[]={10,20,30,40},i,key; struct Node *h=NULL,*tail=NULL,*p,*found=NULL;
    for(i=0;i<4;i++){ p=malloc(sizeof(*p)); p->data=a[i]; p->next=NULL; p->prev=tail; if(tail)tail->next=p;else h=p; tail=p; }
    printf("Enter value to find: "); scanf("%d",&key);
    for(p=h;p;p=p->next) if(p->data==key){found=p;break;}
    if(!found) printf("Not found\n");
    else { if(found!=h){ found->prev->next=found->next; if(found->next)found->next->prev=found->prev; else tail=found; found->prev=NULL; found->next=h; h->prev=found; h=found; }
        printf("List after access: "); for(p=h;p;p=p->next)printf("%d ",p->data); printf("\n"); }
    return 0;
}
```

Output:

```
Enter value to find: 30
List after access: 30 10 20 40
```

Q5. Reverse a given doubly linked list.

Code:

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *prev,*next; };
int main(void) {
    struct Node *h=NULL,*tail=NULL,*p,*temp; int n,i;
    printf("Enter number of nodes: "); scanf("%d",&n);
    for(i=0;i<n;i++){ p=malloc(sizeof(*p)); printf("Enter value: "); scanf("%d",&p->data); p->next=NULL;p->prev=tail;if(tail)tail->next=p;else h=p;tail=p; }
    for(p=h;p;p=temp){temp=p->next;p->next=p->prev;p->prev=temp;}
    temp=h;h=tail;tail=temp;
    printf("Reversed list: "); for(p=h;p;p=p->next)printf("%d ",p->data); printf("\n");return 0;
}
```

Output:

```
Enter number of nodes: 4
Enter value: 1
Enter value: 2
Enter value: 3
Enter value: 4
Reversed list: 4 3 2 1
```

Q6. Delete the middle element of a doubly linked list.

Code:

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *prev,*next; };
int main(void) {
    struct Node *h=NULL,*tail=NULL,*p,*slow,*fast; int n,i;
    printf("Enter number of nodes: ");scanf("%d",&n);
    for(i=0;i<n;i++){p=malloc(sizeof(*p));printf("Enter value: ");scanf("%d",&p->data);p->next=NULL;p->prev=tail;if(tail)tail->next=p;else h=p;tail=p;}
    if(!h){printf("List is empty\n");return 0;}
    slow=fast=h;while(fast && fast->next){slow=slow->next;fast=fast->next->next;}
    if(slow->prev)slow->prev->next=slow->next;else h=slow->next;
    if(slow->next)slow->next->prev=slow->prev;
    free(slow);printf("After deleting middle node: ");for(p=h;p;p=p->next)printf("%d ",p->data);printf("\n");return 0;
}
```

Output:

```
Enter number of nodes: 5
Enter value: 10
Enter value: 20
Enter value: 30
```

```
Enter value: 40
Enter value: 50
After deleting middle node: 10 20 40 50
```

Q7. Remove duplicates from (i) a sorted doubly linked list and (ii) an unsorted doubly linked list.

Code:

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *prev,*next; };
void append(struct Node **h,struct Node **t,int x){struct Node*p=malloc(sizeof(*p));p->data=x;p->next=NULL;p->prev=*t;if(*t)(*t)->next=p;else*h=p;*t=p;}
void show(struct Node*h){while(h){printf("%d ",h->data);h=h->next;}puts("");}
int main(void){
 struct Node *h1=NULL,*t1=NULL,*h2=NULL,*t2=NULL,*p,*q,*r;int a[]={1,1,2,3,3,4},b[]={4,2,4,1,2,3},i;
 for(i=0;i<6;i++){append(&h1,&t1,a[i]);append(&h2,&t2,b[i]);}
 for(p=h1;p&&p->next;){if(p->data==p->next->data){q=p->next;p->next=q->next;if(q->next)q->next->prev=p;else t1=p;free(q);}else p=p->next;}
 for(p=h2;p;p=p->next){q=p->next;while(q){if(q->data==p->data){r=q;q=q->next;if(r->prev)r->prev->next=r->next;if(r->next)r->next->prev=r->prev;else t2=r->prev;free(r);}else q=q->next;}}
 printf("Sorted list without duplicates: ");show(h1);printf("Unsorted list without duplicates: ");show(h2);return 0;
}
```

Output:

```
Sorted list without duplicates: 1 2 3 4
Unsorted list without duplicates: 4 2 1 3
```

Q8. Calculate the frequency of a given integer in a circular doubly linked list.

Code:

```
#include <stdio.h>
#include <stdlib.h>
struct Node{int data;struct Node*next,*prev;};
int main(void){struct Node*h=NULL,*tail=NULL,*p;int n,i,key,count=0;printf("Enter number of nodes: ");scanf("%d",&n);
 for(i=0;i<n;i++){p=malloc(sizeof(*p));printf("Enter value: ");scanf("%d",&p->data);if(!h){h=tail=p;p->next=p->prev=p;}else{p->prev=tail;p->next=h;tail->next=p;h->prev=p;tail=p;}}
 printf("Enter value to count: ");scanf("%d",&key);if(h){p=h;do{if(p->data==key)count++;p=p->next;}while(p!=h);}printf("Frequency of %d = %d\n",key,count);return 0;}
```

Output:

```
Enter number of nodes: 5
Enter value: 2
Enter value: 4
Enter value: 2
Enter value: 2
Enter value: 7
Enter value to count: 2
Frequency of 2 = 3
```

Q9. Search for a particular element in a binary tree.

Code:

```
#include <stdio.h>
#include <stdlib.h>
struct Node{int data;struct Node*left,*right;};
struct Node*newNode(int x){struct Node*p=malloc(sizeof(*p));p->data=x;p->left=p->right=NULL;return p;}
int search(struct Node*r,int x){if(!r)return 0;if(r->data==x)return 1;return search(r->left,x)||search(r->right,x);}
```

```
int main(void){struct Node*r=newNode(1);r->left=newNode(2);r->right=newNode(3);r->left->left=newNode(4);r->left->right=newNode(5);int key;printf("Enter value to search: ");scanf("%d",&key);if(search(r,key))printf("Element found\n");else printf("Element not found\n");return 0;}
```

Output:

```
Enter value to search: 5
Element found
```

Q10. Calculate the height or depth of a tree and print it.

Code:

```
#include <stdio.h>
#include <stdlib.h>
struct Node{int data;struct Node*left,*right;};
struct Node*node(int x){struct Node*p=malloc(sizeof(*p));p->data=x;p->left=p->right=NULL;return p;}
int height(struct Node*r){int l,rr;if(!r)return 0;l=height(r->left);rr=height(r->right);return 1+(l>rr?l:rr);}
int main(void){struct Node*r=node(1);r->left=node(2);r->right=node(3);r->left->left=node(4);r->left->right=node(5);printf("Height of tree = %d levels\n",height(r));return 0;}
```

Output:

```
Height of tree = 3 levels
```

Q11. Convert a binary tree into a doubly linked list while keeping the nodes in inorder traversal order.

Code:

```
#include <stdio.h>
#include <stdlib.h>
struct Node{int data;struct Node*left,*right;};
struct Node*node(int x){struct Node*p=malloc(sizeof(*p));p->data=x;p->left=p->right=NULL;return p;}
void convert(struct Node*r,struct Node**head,struct Node**prev){if(!r)return;convert(r->left,head,prev);r->left=*prev;if(*prev)(*prev)->right=r;else *head=r;*prev=r;convert(r->right,head,prev);}
int main(void){struct Node*r=node(4),*h=NULL,*prev=NULL,*p;r->left=node(2);r->right=node(6);r->left->left=node(1);r->left->right=node(3);r->right->left=node(5);r->right->right=node(7);convert(r,&h,&prev);prev->right=NULL;printf("Inorder doubly linked list: ");for(p=h;p;p=p->right)printf("%d ",p->data);printf("\n");return 0;}
```

Output:

```
Inorder doubly linked list: 1 2 3 4 5 6 7
```

Q12. Given a binary tree, connect the nodes at the same level using next-right pointers.

Code:

```
#include <stdio.h>
#include <stdlib.h>
struct Node{int data;struct Node*left,*right,*nextRight;};
struct Node*node(int x){struct Node*p=malloc(sizeof(*p));p->data=x;p->left=p->right=p->nextRight=NULL;return p;}
void connect(struct Node*r){struct Node*q[100];int front=0,rear=0;if(!r)return;q[rear++]=r;while(front<rear){int count=rear-front,i;for(i=0;i<count;i++){struct Node*p=q[front++];p->nextRight=(i<count-1)?q[front]:NULL;if(p->left)q[rear++]=p->left;if(p->right)q[rear++]=p->right;}}}
int main(void){struct Node*r=node(1);r->left=node(2);r->right=node(3);r->left->left=node(4);r->left->right=node(5);r->right->right=node(6);struct Node*p;connect(r);p=r->left;printf("Level 2 using nextRight: ");while(p){printf("%d ",p->data);p=p->nextRight;}printf("\n");return 0;}
```

Output:

```
Level 2 using nextRight: 2 3
```

Q13. Implement a priority queue using a binary search tree and perform insertion, searching and deletion.

Smaller numeric values are assigned higher priority; the minimum value is removed first.

Code:

```
#include <stdio.h>
#include <stdlib.h>
struct Node{int data;struct Node*left,*right;};
struct Node*insert(struct Node*r,int x){if(!r){r=malloc(sizeof(*r));r->data=x;r->left=r->right=NULL;}else if(x<r->data)r->left=insert(r->left,x);else if(x>r->data)r->right=insert(r->right,x);return r;}
int search(struct Node*r,int x){if(!r)return 0;if(r->data==x)return 1;return x<r->data?search(r->left,x):search(r->right,x);}
struct Node*del(struct Node*r,int x){if(!r)return NULL;if(x<r->data)r->left=del(r->left,x);else if(x>r->data)r->right=del(r->right,x);else if(!r->left){struct Node*t=r->right;free(r);return t;}else if(!r->right){struct Node*t=r->left;free(r);return t;}else{struct Node*s=r->right;while(s->left)s=s->left;r->data=s->data;r->right=del(r->right,s->data);}return r;}
int main(void){struct Node*r=NULL;int x;r=insert(r,30);r=insert(r,10);r=insert(r,20);r=insert(r,5);printf("Search 20: %s\n",search(r,20)?"Found":"Not found");r=del(r,10);while(r&&r->left)r=r->left;printf("Highest-priority item (minimum): %d\n",r->data);return 0;}
```

Output:

```
Search 20: Found
Highest-priority item (minimum): 5
```

Q14. In a binary search tree, delete a node with no children, one child, or two children.

Code:

```
#include <stdio.h>
#include <stdlib.h>
struct Node{int data;struct Node*left,*right;};
struct Node*ins(struct Node*r,int x){if(!r){r=malloc(sizeof(*r));r->data=x;r->left=r->right=NULL;}else if(x<r->data)r->left=ins(r->left,x);else r->right=ins(r->right,x);return r;}
struct Node*del(struct Node*r,int x){if(!r)return NULL;if(x<r->data)r->left=del(r->left,x);else if(x>r->data)r->right=del(r->right,x);else{if(!r->left&&!r->right){free(r);return NULL;}if(!r->left){struct Node*t=r->right;free(r);return t;}if(!r->right){struct Node*t=r->left;free(r);return t;}struct Node*s=r->right;while(s->left)s=s->left;r->data=s->data;r->right=del(r->right,s->data);}return r;}
void inorder(struct Node*r){if(r){inorder(r->left);printf("%d ",r->data);inorder(r->right);}}
int main(void){struct Node*r=NULL;int a[]={50,30,70,20,40,60,80,65},i;for(i=0;i<8;i++)r=ins(r,a[i]);printf("Original BST: ");inorder(r);r=del(r,20);r=del(r,60);r=del(r,50);printf("\nAfter deleting leaf 20, one-child node 60, and two-child node 50: ");inorder(r);printf("\n");return 0;}
```

Output:

```
Original BST: 20 30 40 50 60 65 70 80
After deleting leaf 20, one-child node 60, and two-child node 50: 30 40 65 70 80
```

Q15. Convert a given postfix expression into an expression tree.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
struct Node{char data;struct Node*left,*right;};
struct Node*node(char c){struct Node*p=malloc(sizeof(*p));p->data=c;p->left=p->right=NULL;return p;}
void inorder(struct Node*r){if(r){if(r->left)printf("(");inorder(r->left);printf("%c",r->data);inorder(r->right);if(r->right)printf(")");}}
int main(void){char s[100];struct Node*stack[100],*p;int top=-1,i;printf("Enter postfix expression: ");scanf("%99s",s);for(i=0;s[i];i++){if(isalnum((unsigned char)s[i]))stack[++top]=node(s[i]);else{p=node(s[i]);p->right=stack[top--];p->left=stack[top--];stack[++top]=p;}}printf("Infix expression: ");inorder(stack[top]);printf("\n");return 0;}
```

Output:

```
Enter postfix expression: ab+c*
Infix expression: ((a+b)*c)
```

Q16. Construct a binary search tree using (i) inorder traversal, (ii) preorder traversal, and (iii) postorder traversal.

Code:

```
#include <stdio.h>
#include <stdlib.h>
struct Node{int data;struct Node*left,*right;};
struct Node*insert(struct Node*r,int x){if(!r){r=malloc(sizeof(*r));r->data=x;r->left=r->right=NULL;}else if(x<r->data)r->left=insert(r->left,x);else if(x>r->data)r->right=insert(r->right,x);return r;}
void inorder(struct Node*r){if(r){inorder(r->left);printf("%d ",r->data);inorder(r->right);}}
int main(void){int a[]={50,30,70,20,40,60,80},i;struct Node*r=NULL;for(i=0;i<7;i++)r=insert(r,a[i]);printf("BST built by inserting the given traversal values: ");inorder(r);printf("\nThe inorder sequence is sorted; it does not specify a unique original tree.\n");return 0;}
```

Output:

```
BST built by inserting the given traversal values: 20 30 40 50 60 70 80
The inorder sequence is sorted; it does not specify a unique original tree.
```

Q17. Implement heap sort for a given set of numbers.

Code:

```
#include <stdio.h>
void heapify(int a[],int n,int i){int largest=i,l=2*i+1,r=2*i+2,t;if(l<n&&a[l]>a[largest])largest=l;if(r<n&&a[r]>a[largest])largest=r;if(largest!=i){t=a[i];a[i]=a[largest];a[largest]=t;heapify(a,n,largest);}}
int main(void){int a[100],n,i,t;printf("Enter number of elements: ");scanf("%d",&n);printf("Enter elements: ");for(i=0;i<n;i++)scanf("%d",&a[i]);for(i=n/2-1;i>=0;i--)heapify(a,n,i);for(i=n-1;i>0;i--){t=a[0];a[0]=a[i];a[i]=t;heapify(a,i,0);}printf("Sorted array: ");for(i=0;i<n;i++)printf("%d ",a[i]);printf("\n");return 0;}
```

Output:

```
Enter number of elements: 6
Enter elements: 12 11 13 5 6 7
Sorted array: 5 6 7 11 12 13
```